Managing the Data of a Music Server

This repository contains the code for managing all the data of a music server, including transferring music from a computer to a server and registering it in a database.

Disclaimer & Context (Skippable)

This project is both a learning process and a way to achieve a specific goal. As a result, some parts of the code were made with the assistance of AI or external sources, such as YouTube tutorials and official documentation. The goals of the project were to put my knowledge of databases and SQLite3 into practice, and to fully develop a solid way to manage my private music app.

As my native language is Spanish, some variable names, comments, or documentation may be poorly translated or contextually confusing in English. This document, and the explanations within, aim to fully clarify the project's logic and structure to compensate for any language inconsistencies found in the code itself.

If you have any questions or require clarification on the code, please feel free to contact me at: mcelada04@gmail.com.

Server (Not the code)

Setting up the server and the streaming app was really complicated. I didn't understand half of what I did at first, as I was just following YouTube tutorials to create the server and official documentation for Liquidsoap. Below, I will explain a bit about how it works, focusing on the workflow I followed and everything I learned that could be useful for other projects.

The first step was to install Linux Server on an old computer so it could function as a server. The main PC connects to the server over the network via an SSH connection (ssh username@ip), making it easier to manipulate. Next, you can use Bash to create folders and organize the storage space needed for the project before setting up Docker.

A Docker container is an environment where everything an app needs to run is stored, allowing it to be used on any other computer. This is useful whether you need to switch/move PCs, or if you want to compartmentalize your setup so that mistakes in other code don't affect the whole server, which might store other projects. The Docker setup follows a pattern and needs to contain all the information the project requires: folder paths, exit ports, etc. It can be edited during the project if needed. The Docker container can also be connected to Portainer, a visual environment used

to easily manipulate containers. I created mine through code because that is what the tutorial did, but it can also be done directly in Portainer for an easier setup with less manual configuration.

The next part was configuring Liquidsoap. Unlike Portainer, this tool isn't as useful for other general projects, so I didn't research it beyond what was necessary. The basics to know are that a fail-safe is needed so that if no song is playing or a bug occurs, it falls back to a safe state and doesn't break the music player. It also needs a configuration to read the songs it is supposed to play, a way of knowing the complete path of those songs (in my case, I put the paths of the songs to play into a queue.txt file), and the port it will use to output the stream. I also added an extra port that can receive a "skip" instruction, but this is optional. I guess it is possible to add more ports and instructions, but I won't be investigating further into this matter.

create_db.py

This is a pretty straightforward script. We create a database containing three tables:

1. Songs Table: This contains the song itself and all of its metadata (duration, real name, file path, etc.—what I save at the bottom of the code is entirely personal) with a numerical ID as a primary key and a unique file name constraint.
2. Categories Table: This stores all the possible categories (e.g., sad, classic, movie) a song can have.
3. Song-Category Link Table: This database table links unique song-category pairs so that the exact same song cannot be duplicated and saved twice under the same category. This junction table was created so a song can easily be classified into an unlimited number of categories.

This code only creates the database locally, so it must be executed directly on the server or transferred there after being run on the main PC.

add_songs.py

This is the first script where an SSH connection is used. For all of these scripts, it is assumed that the database is stored on the server but the code will run on a local computer. Therefore, this script and the next one begin the same way: by opening an SSH connection with the server (requiring all the proper credentials) and downloading the database into a temporary file. I read that this is a good practice because it is the easiest way to make changes safely; any mistake made on the local PC will not immediately affect the live database on the server.

The code then proceeds to scan all the song paths in the music_to_pass folder. It opens each file, extracts its metadata, and prepares to update the database. The

TinyTag library is used to read the internal metadata of the audio files, while other libraries like os and datetime gather additional data that I wanted. This part is entirely personal, based on the data I felt would be most relevant. The condition if result is None is simply a way to make sure the song does not already exist in the database before attempting to insert it.

The process labeled "PASS THE FILE FROM PC TO SERVER AND DELETE" is the method I designed to ensure data consistency on the server. The original problem was a synchronization risk: if songs are moved to the server at the exact same time they are registered in the database, a network interruption or crash before the final update could leave files on the server that are missing from the database. Conversely, if files are uploaded after the database is transferred, the same issues could leave records in the database that do not exist on the server.

My solution was to create a tracking file: every time a song is registered in the database, it is also logged in a temporary text file. After the database is successfully updated on the server, the script begins transferring the audio files. Each time a song is successfully moved, its name is deleted from the text file. If an error occurs, it is very easy to spot exactly which songs failed to transfer.

Finally, just like in the next script, the SSH connection is closed and the temporary files are deleted.

categorize_songs.py

This script is the longest, but most of it consists of user instructions and inputs designed to make classifying music much easier.

First, you decide which type of songs to classify (complete with a fail-safe and default values). Then, the target song is appended 10 times to queue.txt, which is uploaded to the server, and two "skip" commands are sent (this forces the system to skip the current fail-safe filler tracks so the song being classified starts playing immediately).

For every song, there are five menu options available:

- exit: Skips the current song.
- sup_exit: Ends the classification session and closes the script.
- create: Creates a brand-new category.
- view: Displays all existing categories.
- Entering any other category name: Updates the database to assign the current song to that category.

Afterward, the connections are closed and the temporary files are cleaned up.

The prompt_toolkit library is utilized here to help the user view and select all the categories much more easily.